

3 Fetch API - Appels HTTP

3.1 Qu'est-ce que fetch() ?

Information

`fetch()` est une fonction JavaScript native pour faire des requêtes HTTP (GET, POST, PUT, DELETE).

Syntaxe basique :

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

3.2 Fetch avec useEffect - Pattern Complet

IMPORTANT

Pattern - À connaître par coeur :

```
import { useState, useEffect } from 'react';

function DataComponent() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => {
        if (!response.ok) {
          throw new Error('Erreur réseau');
        }
        return response.json();
      })
      .then(data => {
        setData(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, []);

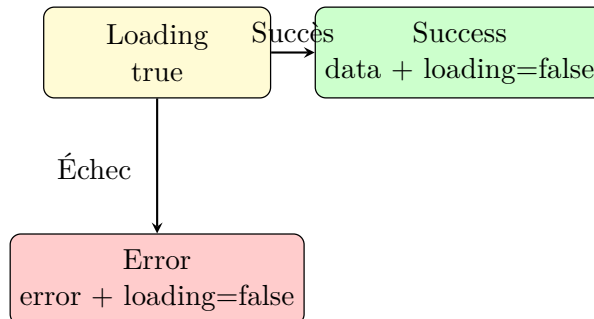
  if (loading) return <p>Chargement...</p>;
  if (error) return <p>Erreur : {error}</p>;

  return (
    <div>
      {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
    </div>
  );
}
```

3.3 Les 3 States Essentiels

Pour gérer un appel API, on a besoin de **3 states** :

1. **data** : Les données reçues de l'API
2. **loading** : Est-ce que la requête est en cours ?
3. **error** : Y a-t-il eu une erreur ?



3.4 Promises - .then() et .catch()

`fetch()` retourne une **Promise** (promesse).

```
fetch(url)
  .then(response => {
    // 1. Réponse brute du serveur
    console.log(response);
    return response.json(); // Convertir en JSON
  })
  .then(data => {
    // 2. Données JSON parsées
    console.log(data);
  })
  .catch(error => {
    // 3. En cas d'erreur (réseau, parsing, etc.)
    console.error(error);
  });
```

Listing 7 – Anatomie d'une Promise

Ordre d'exécution :

1. `fetch(url)` - Envoie la requête
2. Premier `.then()` - Reçoit la réponse
3. Deuxième `.then()` - Reçoit les données JSON
4. `.catch()` - Attrape les erreurs

3.5 `async/await` - Syntaxe Moderne

Une alternative plus lisible aux `.then()`.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      setLoading(true);
      const response = await fetch('https://api.example.com/data');

      if (!response.ok) {
        throw new Error('Erreur HTTP');
      }
    }
  }
});
```

```
    const data = await response.json();
    setData(data);
  } catch (error) {
    setError(error.message);
  } finally {
    setLoading(false);
  }
};

fetchData();
}, []);
```

Listing 8 – Fetch avec async/await

Astuce

async/await vs .then() :

Les deux sont équivalents. **async/await** est plus moderne et lisible, mais **.then()** est plus compact.

Pour les **examens papier**, les deux syntaxes sont acceptées.

3.6 Vérification response.ok

Attention - Erreur Fréquente

Erreur fréquente :

fetch() ne rejette PAS la Promise en cas d'erreur HTTP (404, 500, etc.). Il faut vérifier **response.ok** manuellement.

```
// INCOMPLET
fetch(url)
  .then(res => res.json()) // Ne détecte pas erreurs HTTP !

// CORRECT
fetch(url)
  .then(res => {
    if (!res.ok) {
      throw new Error('HTTP error! status: ${res.status}');
    }
    return res.json();
  })
```

4 Patterns Complets avec useEffect

4.1 Fetch Liste d'Utilisateurs

```
import { useState, useEffect } from 'react';

function UsersList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => {
        if (!response.ok) {
          throw new Error('Erreur de chargement');
        }
        return response.json();
      })
      .then(data => {
        setUsers(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <div className="spinner">Chargement...</div>;
  }

  if (error) {
    return <div className="error">Erreur : {error}</div>;
  }

  return (
    <div>
      <h2>Liste des Utilisateurs</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>
            {user.name} - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Listing 9 – Exemple complet - Liste utilisateurs

4.2 Fetch avec Dépendance (Filtre)

```
function FilteredPosts() {
  const [posts, setPosts] = useState([]);
  const [userId, setUserId] = useState(1);
  const [loading, setLoading] = useState(false);
```

```

useEffect(() => {
  setLoading(true);

  fetch('https://jsonplaceholder.typicode.com/posts?userId=${userId}')
    .then(res => res.json())
    .then(data => {
      setPosts(data);
      setLoading(false);
    });
}, [userId]); // Re-fetch quand userId change

return (
  <div>
    <select value={userId} onChange={(e) => setUserId(e.target.value)}>
      <option value="1">User 1</option>
      <option value="2">User 2</option>
      <option value="3">User 3</option>
    </select>

    {loading && <p>Chargement...</p>}

    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  </div>
);
}

```

Listing 10 – Fetch basé sur un filtre

4.3 Fetch Détail avec ID

```

function UserDetails({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Vérifier que userId existe
    if (!userId) return;

    setLoading(true);

    fetch('https://jsonplaceholder.typicode.com/users/${userId}')
      .then(res => res.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      });
  }, [userId]); // Re-fetch si userId change

  if (loading) return <p>Chargement...</p>;
  if (!user) return <p>Aucun utilisateur</p>;

  return (
    <div>
      <h3>{user.name}</h3>
      <p>Email: {user.email}</p>
      <p>Téléphone: {user.phone}</p>
    </div>
  );
}

```

```

    </div>
  );
}

```

Listing 11 – Fetch détail basé sur ID

4.4 Spinner de Chargement

```

function Spinner() {
  return (
    <div style={{
      display: 'flex',
      justifyContent: 'center',
      padding: '20px'
    }}>
      <div style={{
        border: '4px solid #f3f3f3',
        borderTop: '4px solid #3498db',
        borderRadius: '50%',
        width: '40px',
        height: '40px',
        animation: 'spin 1s linear infinite'
      }} />
    </div>
  );
}

// CSS à ajouter
@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

```

Listing 12 – Composant Spinner réutilisable

4.5 Message d'Erreur

```

function ErrorMessage({ error, onRetry }) {
  return (
    <div style={{
      backgroundColor: '#fee',
      border: '1px solid #fcc',
      padding: '15px',
      borderRadius: '4px',
      color: '#c33'
    }}>
      <p><strong>Erreur :</strong> {error}</p>
      {onRetry && (
        <button onClick={onRetry}>Réessayer</button>
      )}
    </div>
  );
}

// Utilisation
if (error) {
  return <ErrorMessage error={error} onRetry={fetchData} />;
}

```